

Lazy Evaluation

CSC345: Programming Languages & Paradigms

Introduction

- Haskell is lazy
- The opposite of *lazy evaluation* is strict evaluation

Strict Evaluation

- *Strategy*: function arguments are completely evaluated *before* passing them to a function

Example:

(function definition)

`f x y = x + 5`

(function call)

`> f 3 (2+2)`

`f 3 4`

`8`

(function call)

`> f 3 (17^33218)`

`...`

- Observe that `f` never uses 2nd argument, so any computational work on evaluating the 2nd is wasted effort.

Another Example

```
inc :: Int -> Int  
inc n = n + 1
```

Strict Evaluation

```
> inc (2*3)  
{applying *}  
inc 6  
{applying inc}  
6+1  
7
```

Lazy Evaluation

```
> inc (2*3)  
{applying inc}  
(2*3) + 1  
{applying *}  
6+1  
{applying +}  
7
```

- Observe that in this example, the evaluation order *did not* affect the final result.
- **Prompt:** why is strict evaluation useful?
 - Why is it used in Java?
 - (Haskell uses lazy evaluation.)

Benefits of Strict Evaluation

- Easy to predict when, and in what order, evaluations will happen.
- Most/all programming languages with strict evaluation specify that function arguments should be evaluated left-to-right.
- *Java Example:* `f (g (), h ())`
 1. Evaluate and compute what `g` returns
 2. Evaluate and compute what `h` returns
 3. Pass argument values to `f`, regardless of whether `f` actually uses them

Example 3

Example with variables, states, and side effects:

Starting value / state:

$n = 0$

Imperative Expression: “adds the current value of the variable n to the result of changing its value to one”

$n + (n = 1)$
{applying n }
 $0 + (n = 1)$
{applying $=$ }
 $0 + 1$
{applying $+$ }
1

$n + (n = 1)$
{applying $=$ }
 $n + 1$
{applying n }
 $1 + 1$
{applying $+$ }
2

- Observe difference in final value!
 - The decision of whether to first perform the left-side addition or the right-side assignment makes a big difference!

Strict Evaluation

- Strict Evaluation precisely specifies how function arguments and operands are evaluated
 - Precise order is necessary if “side effects” are allowed in a P/L.
- But, there are no side effects in Haskell.
 - Big idea: Any two different way of evaluating the same expression will always produce the same final value!
 - The time at which a function is applied to an argument never affects the result.

Haskell: a lazy, functional language

- Lazy Evaluation strategy:
 - Evaluation of function arguments is *delayed as long as possible*
 - Arguments are not evaluated until it becomes necessary
 - If an evaluation is not necessary, function arguments are passed as an “unevaluated expression”

Example:

(function call)

(function definition)

`f x y = x + 5`

`> f 3 (17^33218)`

8

- Second argument will be packaged up as an “unevaluated expression”
- Will be thrown away by the Haskell garbage collector since `f` never uses it
- **Prompt:** when is it necessary to evaluate an expression?

When is it necessary to evaluate an expression?

Some scenarios:

Scenario 1: A function uses an argument doesn't need to evaluate it.

- Function doesn't need to know anything about the argument

```
f1 :: a -> (a,a)  
f1 n = (n,n)
```

- The unevaluated expression is used to compose a tuple

When is it necessary to evaluate an expression?

Some scenarios:

Scenario 2: Pattern matching requires a function to know something about its argument.

- Is the argument constructed from the **Nothing** or **Just** data constructor?

```
f2 :: Maybe a -> [a]
f2 Nothing = []
f2 (Just n) = [n]
```

- Function must evaluate the incoming argument to perform the pattern matching, but only just enough to perform the pattern matching.

(function call)

```
> f2 (Just 5)
[5]
```

(function call)

```
> f2 (Just (17^33218))
[17^33218]
```

17³³²¹⁸ can remain as an unevaluated expression!

When is it necessary to evaluate an expression?

Some scenarios:

Scenario 3: Guards

- Guards are evaluated in-turn

```
f3 :: Int -> Int -> Int -> Int
f3 m n p
  | m >= n    = m
  | otherwise = p
```

- Need to evaluate **m** and **n**
- **p** is potentially returned as an unevaluated expression

When is it necessary to evaluate an expression?

Some scenarios:

Scenario 4: Local Definitions (**where** construct)

Example

> f 3 5

```
f m n
| notNil xs = front xs
| otherwise = n
where
  xs = [m..n]

front (x:y:zs) = x+y
front [x] = x

notNil [] = False
notNil (_:_) = True
```

Evaluation Order

1. Evaluation in Haskell is performed from the outside to the inside

```
> f 3 (g 1 2)
{applying f}
3 * (g 1 2)
{applying g}
3 * (1 + 2)
3 * 3
9
```

```
g :: Int -> Int -> Int
g x y = x + y
```

```
f :: Int -> Int -> Int
f x y = x * y
```

Evaluation Order

2. Otherwise, evaluation is left-to-right.

```
> (f 1 2) + (f 1 3)
{applying leftmost f}
(1 * 2) + (f 1 3)
2 + (f 1 3)
{applying rightmost f}
2 + (1 * 3)
2 + 3
5
```

```
g :: Int -> Int -> Int
g x y = x + y
```

```
f :: Int -> Int -> Int
f x y = x * y
```

No Duplicated Evaluations

- Calculations are performed using a graph data structure
 - Result: a duplicated argument is never evaluated more than once.

$h \ x \ y = x + x$

> h (9-3) (h 34 3)

{evaluate from the outside}

(9-3) + (9-3)

(two current unevaluated expressions)

{evaluate from the left}

6 + 6

12



